

动态词向量预训练模型

目录

CONTENTS

- 1 词向量——从静态到动态**
- 2 基于语言模型的动态词向量预训练**

目录

CONTENTS

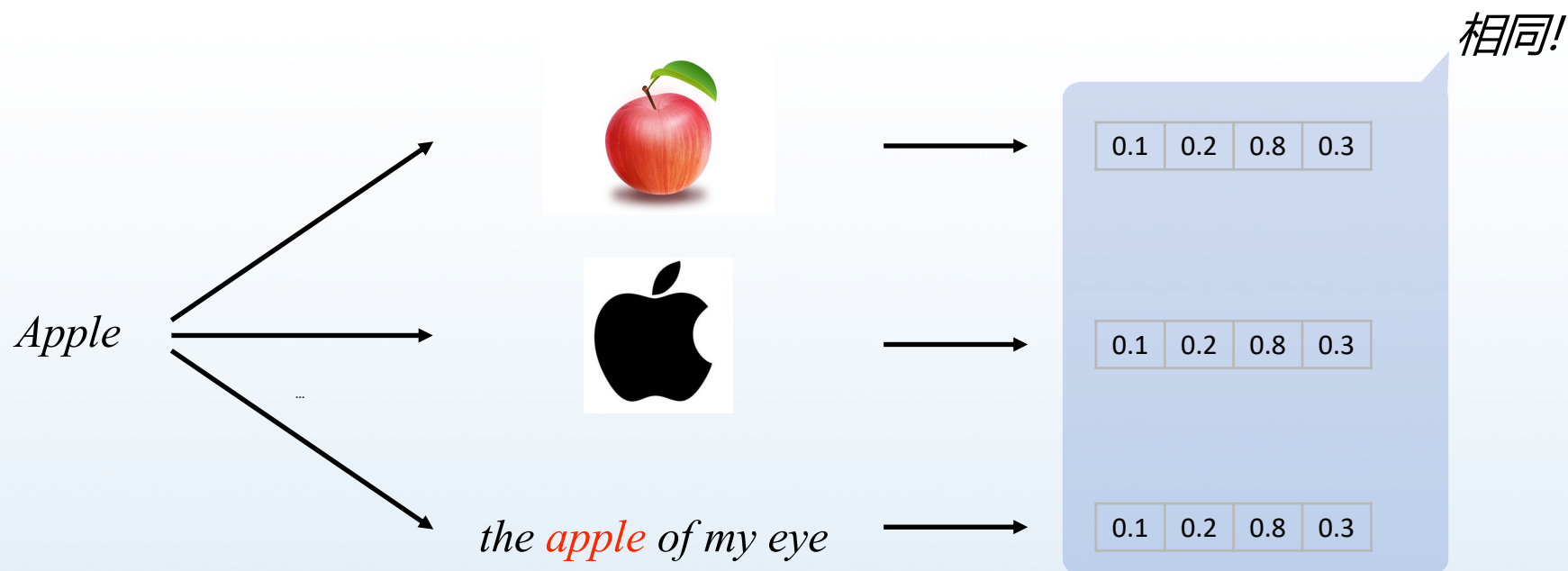
1 词向量——从静态到动态

2 基于语言模型的动态词向量预训练

词向量——从静态到动态

静态词向量的问题

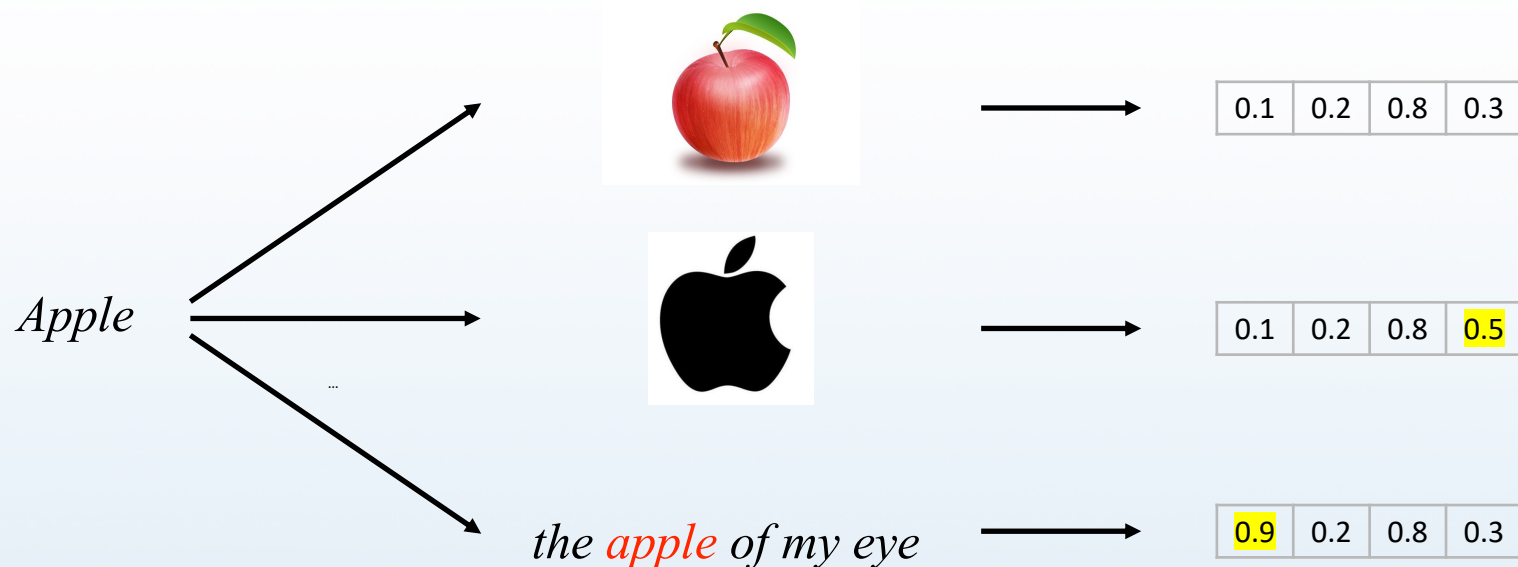
- 很多词包含多种语义信息，静态词向量无法解决“一词多义”的表示问题



词向量——从静态到动态

□ 静态词向量的问题

□ 词向量应根据其所处的上下文的不同而发生改变

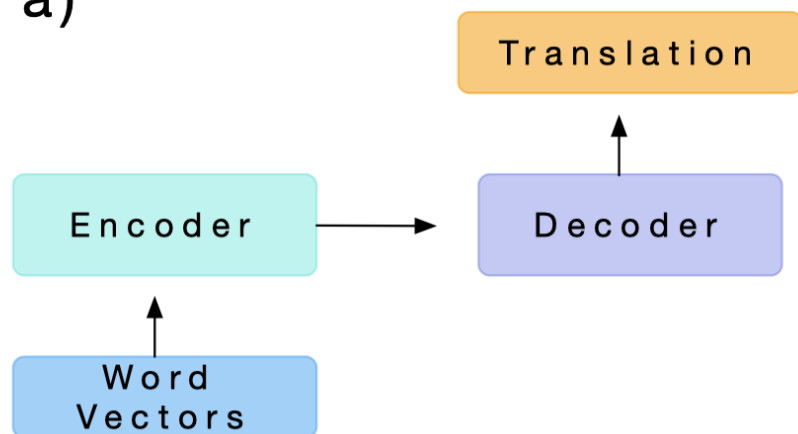


词向量——从静态到动态

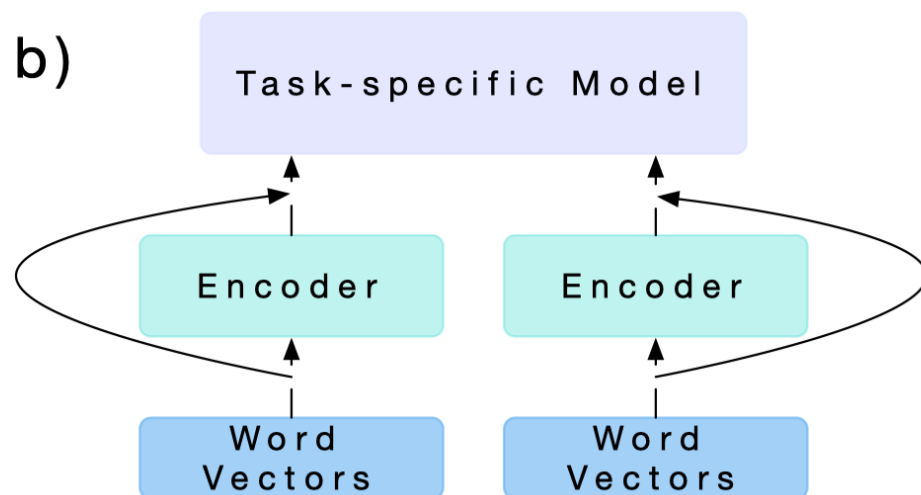
❑ CoVe (Contextualized Word Vectors)

- ❑ 首次提出使用上下文相关的文本表示，即每个token的向量表示不唯一
- ❑ 主要思想：将神经机器翻译(NMT)的表示迁移到通用NLP任务上

a)



b)



词向量——从静态到动态

❑ CoVe (Contextualized Word Vectors)

❑ 训练阶段：训练一个神经机器翻译模型（NMT）

❑ Encoder: 两层双向的LSTM模型

❑ 给定一个源语言句子 w^x 和目标语言句子 w^z

$$h = \text{MT-LSTM}(\text{GloVe}(w^x))$$

❑ Decoder:

❑ 两层、单向的LSTM

$$h_t^{\text{dec}} = \text{LSTM}([z_{t-1}; \tilde{h}_{t-1}], h_{t-1}^{\text{dec}})$$

❑ 基于注意力机制的解码器

$$\alpha_t = \text{softmax}(H(W_1 h_t^{\text{dec}} + b_1))$$

$$\tilde{h}_t = [\tanh(W_2 H^\top \alpha_t + b_2; h_t^{\text{dec}})]$$

❑ 输出层

$$p(\hat{w}_t^z | X, w_1^z, \dots, w_{t-1}^z) = \text{softmax}(W_{\text{out}} \tilde{h}_t + b_{\text{out}})$$

词向量——从静态到动态

□ CoVe (Contextualized Word Vectors)

□ 如何应用在下游任务

□ 对于给定的一个句子 w ，通过如下方式获得CoVe表示

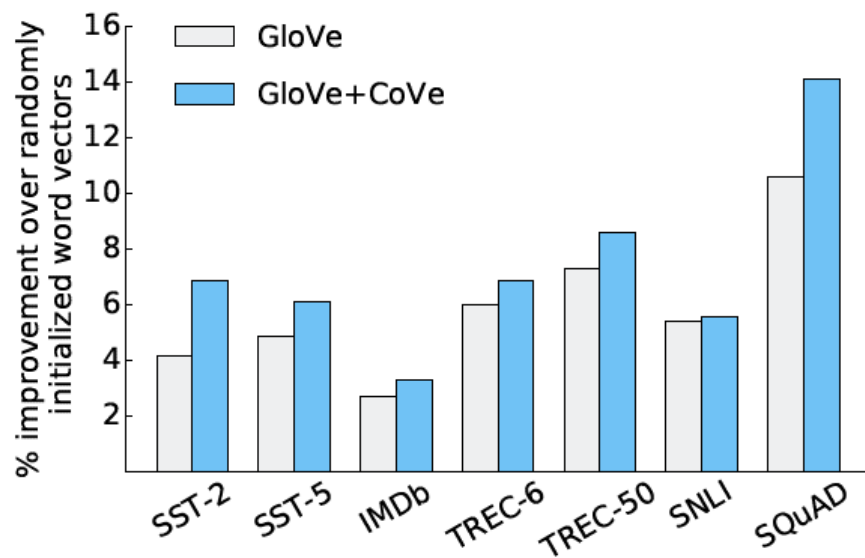
$$\text{CoVe}(w) = \text{MT-LSTM}(\text{GloVe}(w))$$

□ 可与传统的GloVe等词向量拼接，增强语义表示能力

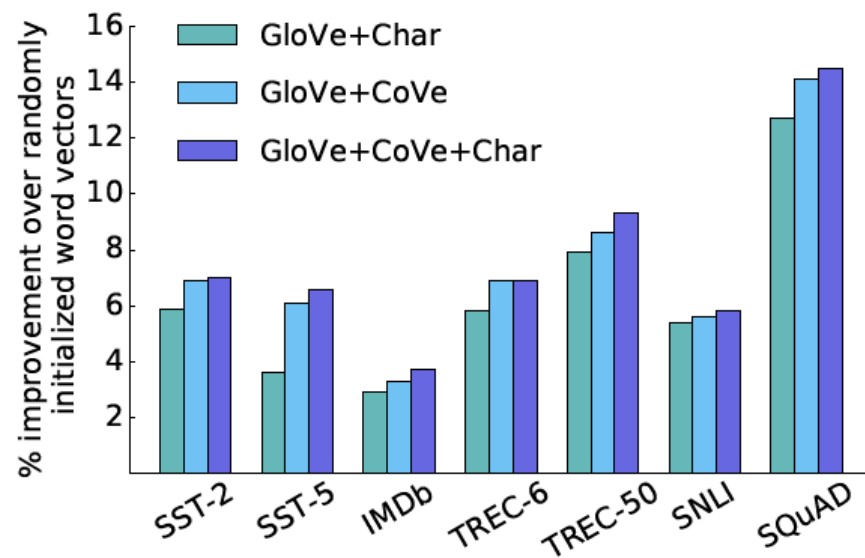
$$\tilde{w} = [\text{GloVe}(w); \text{CoVe}(w)]$$

词向量——从静态到动态

□ CoVe实验结果



(a) CoVe and GloVe



(b) CoVe and Characters

目录

CONTENTS

1

词向量——从静态到动态

2

基于语言模型的动态词向量预训练

基于语言模型的动态词向量预训练

❑ CoVe存在的问题

训练依赖于双语平行语料

- ❑ 训练神经机器翻译模型需要双语平行语料，获取难度较高
- ❑ 相比单语语料，覆盖的领域也相对有限，通用性一般

单独使用效果一般，性价比不高

- ❑ 实验结果表明单独使用CoVe的效果一般
- ❑ 需要搭配传统静态词向量才能获得较为显著的性能提升

基于语言模型的动态词向量预训练

□ 双向语言模型BiLM

- 双向语言模型：对于给定的一段输入文本 $w_1 w_2 \cdots w_n$ ，BiLM从前向(从左到右)和后向(从右到左)两个方向同时建立语言模型
- 对于文本中任一时刻的词 w_t ，可以同时获得其分别基于左侧上下文信息和右侧上下文信息的表示
- 具体：
 - 模型首先对每个词单独编码，这一过程是上下文无关的，主要利用了词内部的字符序列信息。
 - 基于编码后的词表示序列，模型使用两个不同方向的多层LSTM分别计算每一时刻词的前向、后向隐含层表示，也就是上下文相关的词向量表示。
 - 利用该表示，模型预测每一时刻的目标词。
 - 对于前向语言模型：t时刻的目标词为 w_{t+1}
 - 对于后向语言模型：t时刻的目标词 w_{t-1}

基于语言模型的动态词向量预训练

□ 双向语言模型BiLM

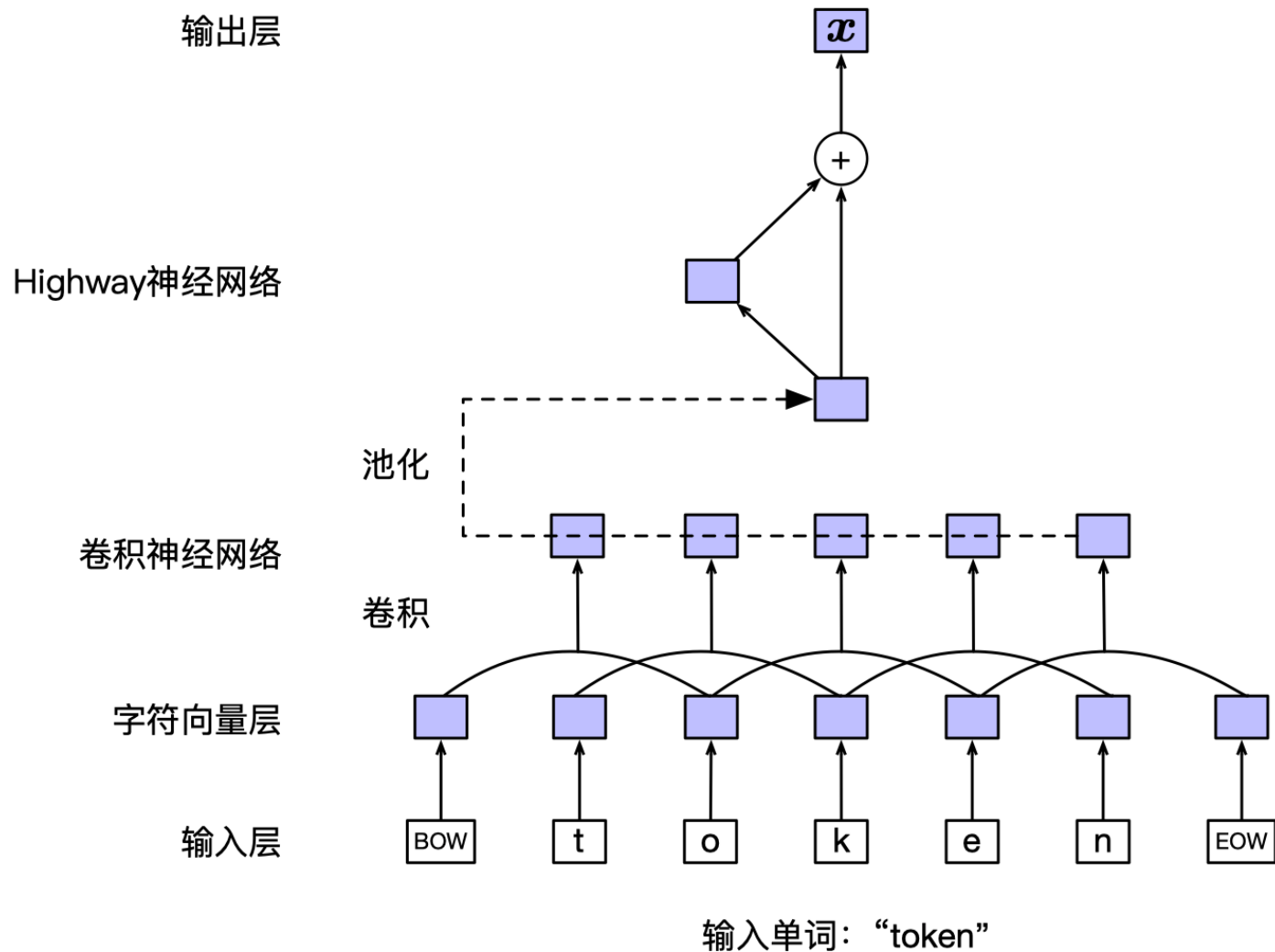
□ 步骤

- ① 输入层表示
- ② 前向语言模型
- ③ 后向语言模型

基于语言模型的动态词向量预训练

□ 双向语言模型BiLM

① 输入层表示



双向语言模型BiLM

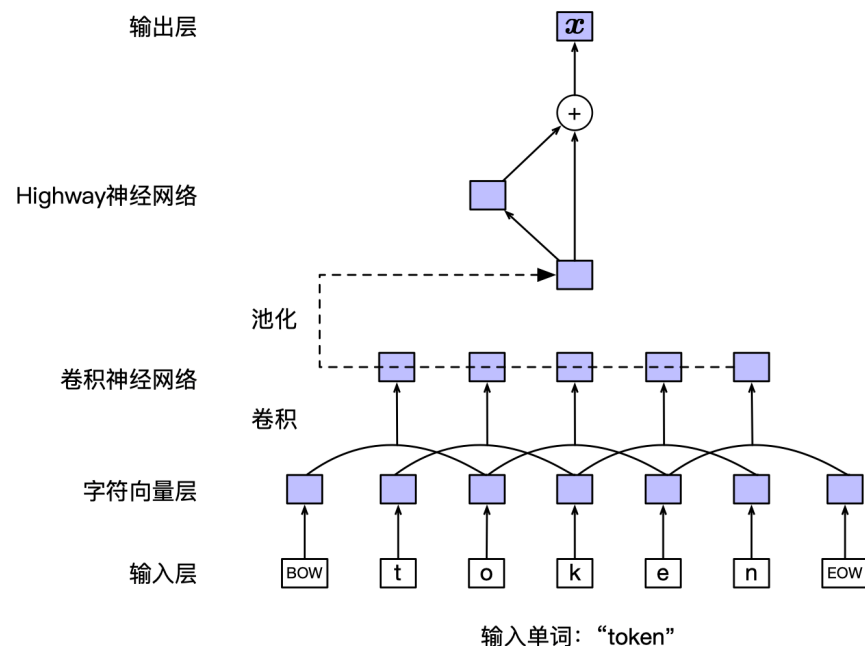
输入层表示

- ELMo模型采用基于字符组合的神经网络表示输入文本中的每个词
- 目的：减小未登录词对模型的影响
- 首先，字符向量层将输入层中的每个字符转换为向量表示

- 假设 w_t 由字符序列 $c_1 c_2 \dots c_l$ 构成，对于其中的每个字符 c_i ，可以表示为

$$v_{c_i} = E^{char} e_{c_i}$$

- 字符向量矩阵： $E^{char} \in R^{d^{char} \times |V^{char}|}$
- 所有字符集合： $|V^{char}|$
- 字符向量维度： d^{char}
- 字符 c_i 的独热编码： e_{c_i}



- 记 w_t 中所有字符向量组成的矩阵为：

$$C_t = [v_{c_1}; v_{c_2}; \dots v_{c_l}]$$

$$C_t \in R^{d^{char} \times l}$$

双向语言模型BiLM

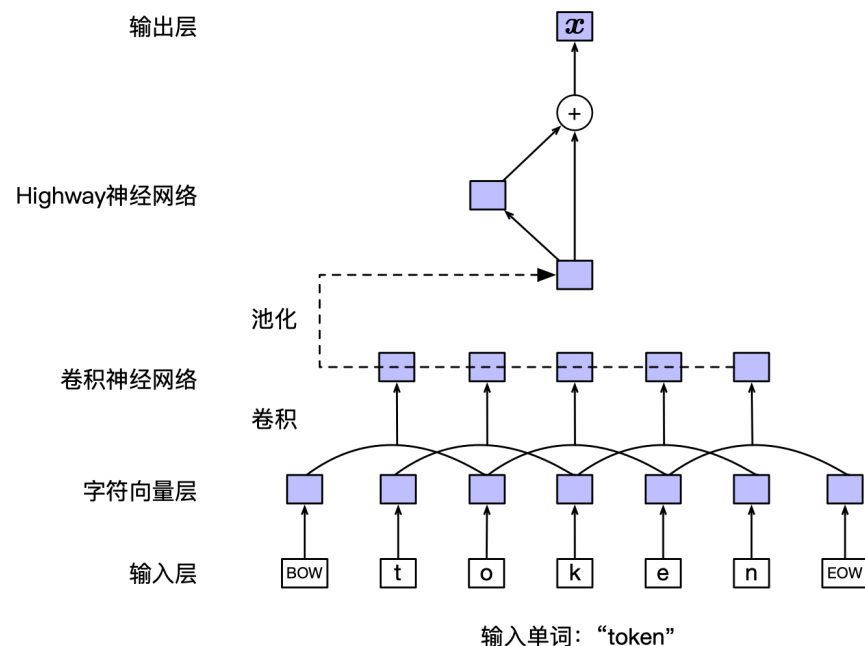
输入层表示

接下来，利用CNN对字符级向量表示序列进行语义组合

卷积：使用多个不同大小（宽度）的卷积核，可以利用不同粒度的字符级上下文信息，并得到相应的隐含层向量表示，这些隐含层向量的维度由每个卷积核对应的输出通道个数确定

池化

拼接：拼接这些向量，就得到了每一位置的卷积输出

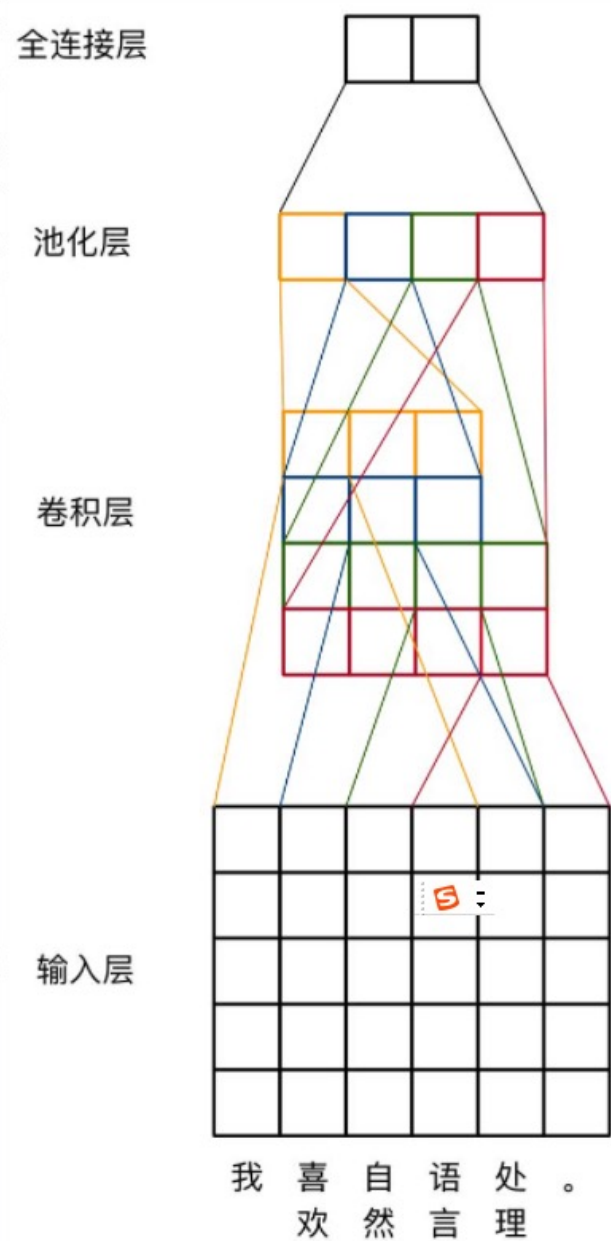


双向语言模型BiLM

□输入层表示

□例如：

- 假设使用7个一维卷积核，宽度分别为{1, 2, 3, 4, 5, 6, 7}的，
- 对应的输出通道数量：分别为{32, 32, 64, 128, 256, 512, 1024}
- 输出向量 f_t 的维度：2048



双向语言模型BiLM

输入层表示

模型使用两层 Highway 神经网络对卷积神经网络输出作进一步变换，得到最终的词向量表示 x_t 。

Highway 神经网络在输入与输出之间直接建立“通道”，使得输出层可以直接将梯度回传至输入层

避免梯度爆炸或弥散的问题

单层Highway神经网络的具体计算方式如下

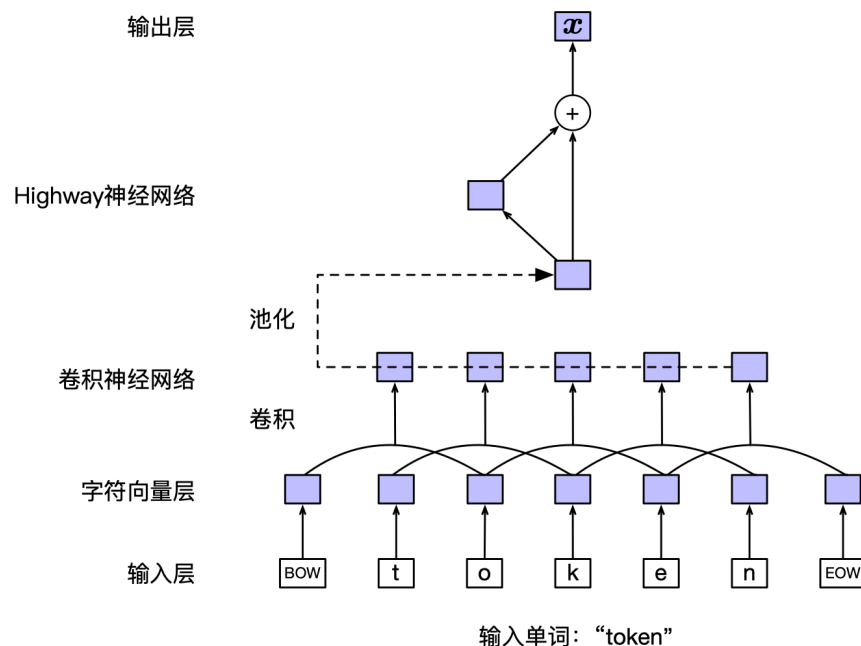
$$x_t = g \odot f_t + (1 - g) \odot \text{ReLU}(W f_t + b)$$

g 为门控向量，其以 f_t 为输入，经线性变换后通过Sigmoid函数(σ)计算得到：

$$g = \sigma(W^g f_t + b^g)$$

W^g 与 b^g 为门控网络中的线性变换矩阵与偏置向量

Highway神经网络的输出实际上是输入层与隐含层的线性插值结果



基于语言模型的动态词向量预训练

□ 双向语言模型 BiLM $v_{c_i} = E^{\text{char}} e_{c_i}$

□ 输入表示层 $x_t = g \odot f_t + (1 - g) \odot \text{ReLU}(W f_t + b)$

$$g = \sigma(W^g f_t + b^g)$$

□ 前向语言模型

□ 对于任一时刻目标词的预测，都只依赖于该时刻左侧的上下文信息或者历史

$$P(w_1 w_2 \cdots w_n) = \prod_{t=1}^n P(w_t | x_{1:t-1}; \vec{\theta}^{\text{lstn}}, \theta^{\text{out}})$$

多层堆叠
LSTM的参数

Softmax输出层参数

□ 后向语言模型

□ 后向语言模型只考虑某一时刻右侧的上下文信息

$$P(w_1 w_2 \cdots w_n) = \prod_{t=1}^n P(w_t | x_{t+1:n}; \overleftarrow{\theta}^{\text{lstn}}, \theta^{\text{out}})$$

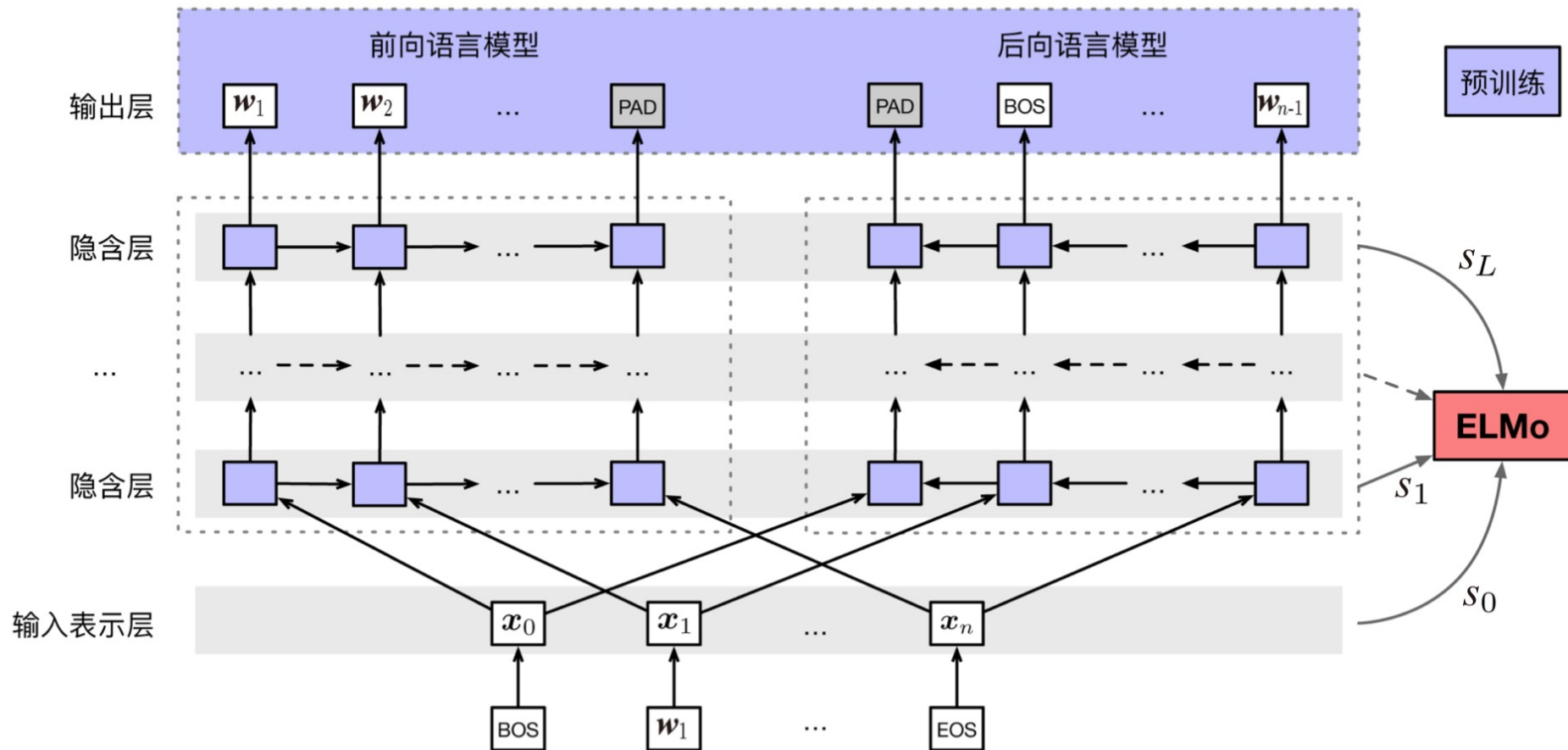
Softmax输出层参数

多层堆叠
LSTM的参数

最大化前向语言模型与后向语言模型的似然函数

基于语言模型的动态词向量预训练

ELMo词向量



基于语言模型的动态词向量预训练

□ELMo词向量

- 在双向语言模型预训练完成后，模型的编码部分（包括输入表示层以及多层堆叠LSTM）便可以用来计算任意文本的动态词向量表示
- 最自然的做法：使用两个LSTM 的最后一层隐含层输出作为词的动态向量表示
- 在ELMo 模型中，不同层次的隐含层向量蕴含了不同层次或粒度的文本信息
 - 例如，越接近顶层的LSTM隐含层表示通常编码了更多的语义信息，而接近底层的隐含层表示（包括输入表示 x ）更偏重于词法、句法信息。
 - 不同的下游任务，对词表示的需求程度有所不同。例如，
 - 阅读理解、自动问答这类任务:对语义信息的需求较高
 - 命名实体识别等任务: 词法、句法信息更重要

基于语言模型的动态词向量预训练

□ ELMo词向量

□ ELMo采取对不同层次的向量表示进行加权平均的机制，为不同的下游任务提供更多的组合自由度

□ 令 $\mathbb{R}_t$ 表示 w_t 的所有中间层状态向量表示构成的集合，则

$$\mathbb{R}_t = \{x_t, h_{t,j} | j = 1, \dots, L\} \quad h_{t,j} = [\overleftarrow{h}_{t,j}; \overrightarrow{h}_{t,j}]$$

两个多层堆叠LSTM中每一层的前向、后向隐含层输出拼接后得到的向量

□ 令 $h_{t,0} = x_t$ ，则ELMo词向量可表示为

$\Psi = \{s^{\text{task}}, \gamma^{\text{task}}\}$: 计算ELMo向量所需的额外参数

$$\text{ELMo}_t = f(\mathbb{R}_t, \Psi) = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} h_{t,j}$$

s^{task} :

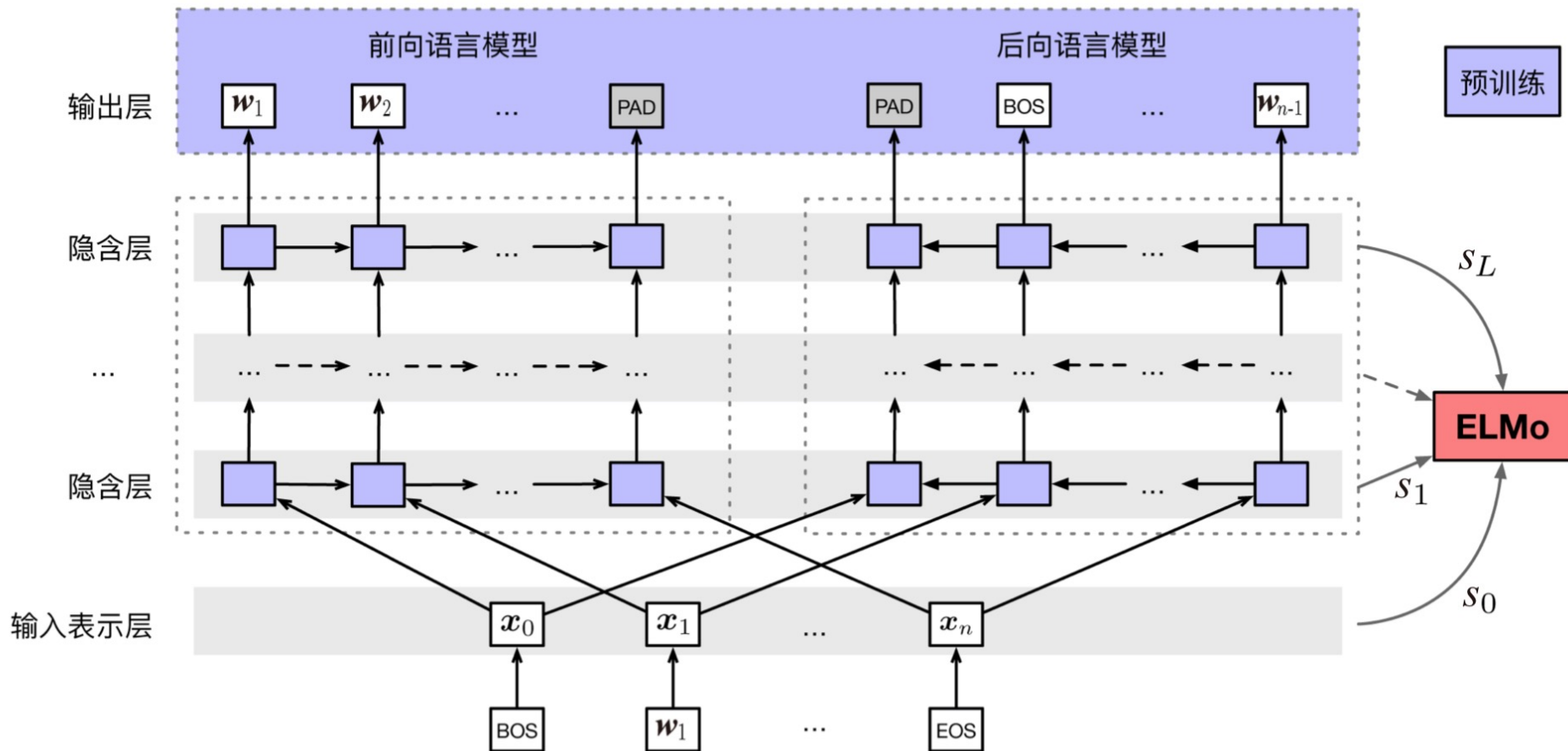
- 表示每个向量的权重
- 反映每一层向量对于目标任务的重要性
- 可由一组参数根据Softmax函数归一化计算得到，该权重向量可在下游任务的训练过程中学习

□ γ^{task} 系数同样与下游任务相关，当ELMo向量与其他向量共同作用时，可以适当地缩放ELMo向量。

□ 将ELMo向量作为词特征用于下游任务时，参数将被“冻结”，不参与更新。

基于语言模型的动态词向量预训练

ELMo词向量



基于语言模型的动态词向量预训练

□ ELMo词向量

- ELMo采取对不同层次的向量表示进行加权平均的机制，为不同的下游任务提供更多的组合自由度

$$\mathbb{R}_t = \{\mathbf{x}_t, \mathbf{h}_{t,j} | j = 1, \dots, L\} \quad \text{ELMo}_t = f(\mathbb{R}_t, \Psi) = \gamma^{\text{task}} \sum_{j=0}^L s_j^{\text{task}} \mathbf{h}_{t,j}$$

□ ELMo特点

- **动态（上下文相关）**：词的ELMo向量表示由其当前上下文决定；
- **鲁棒（Robust）**：ELMo向量表示使用字符级输入，对于未登录词具有强鲁棒性；
- **层次**：ELMo词向量由深度预训练模型中各个层次的向量表示进行组合，为下游任务提供了较大的使用自由度。

基于语言模型的动态词向量预训练

□ 模型实现

□ 数据准备

- 使用清洗后并经过分词等预处理的语料
- 需要同时构建词级别与字符级别的训练语料，并建立相应的词表

□ 双向语言模型

- ELMo模型的核心是双向语言模型
- 编码器部分主要包括基于字符的输入表示层以及前向、后向LSTM 层

□ 训练

- 在数据、模型组件构建完成后，使用实际数据对模型进行训练
- 训练过程将输出每一次迭代后的前向语言模型的困惑度值
- 训练完成后，便可以利用双向语言模型的编码器编码输入文本并获取动态词向量

基于语言模型的动态词向量预训练

□ 模型实现

□ 使用AllenNLP调用ELMo

```
>>> from allennlp.modules.elmo import Elmo, batch_to_ids
>>> options_file = "https://allennlp.s3.amazonaws.com/models/elmo/2
x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2xhighway_options.
json"
>>> weights_file = "https://allennlp.s3.amazonaws.com/models/elmo/2
x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2xhighway_weights.
hdf5"
>>> elmo = Elmo(options_file, weight_file, num_output_representations=1,
dropout=0)
```

```
>>> sentences = [['I', 'love', 'Elmo'], ['Hello', 'Elmo']]
>>> character_ids = batch_to_ids(sentences)
# 输出大小为2*3*50(字符向量维度)的张量
>>> embeddings = elmo(character_ids)
>>> print(embeddings)
```

基于语言模型的动态词向量预训练

□ 模型实现

```
>>> sentences = [['I', 'love', 'Elmo'], ['Hello', 'Elmo']]
```

```
>>> character_ids = batch_to_ids(sentences)
```

```
# 输出大小为2*3*50(字符向量维度)的张量
```

```
>>> embeddings = elmo(character_ids)
```

```
>>> print(embeddings)
```

```
{'elmo_representations':
```

```
[tensor([[[ 0.1474, -0.1475,  0.1376, ...,  0.0270, -0.4051, -0.0498],
          [ 0.2394,  0.0769,  0.4126, ..., -0.1671, -0.1707,  0.3884],
          [-0.7602, -0.4944, -0.5355, ..., -0.0803,  0.0361,  0.1128]],

         [[ 0.2603, -0.4437,  0.2726, ..., -0.0830, -0.1522, -0.1361],
          [-0.7772, -0.4294, -0.2651, ..., -0.0803,  0.0361,  0.1128],
          [ 0.0000,  0.0000,  0.0000, ...,  0.0000,  0.0000,  0.0000]]],
      grad_fn=<CopySlices>)],
```

```
'mask':
```



```
tensor([[[ True,  True,  True],
         [ True,  True, False]])}]
```

基于语言模型的动态词向量预训练

应用与评价

作为下游任务特征

即插即用，可以与静态词向量进行拼接

$$[x_k; \text{ELMo}_k]$$

也可以与隐层输出进行拼接 $[h_k; \text{ELMo}_k]$

右侧给出了利用ELMo实现文本分类的示例代码

```
class ELMoMLP(nn.Module):  
    def __init__(self, elmo, hidden_dim, num_class):  
        super(ELMoMLP, self).__init__()  
        # ELMo预训练编码器，可使用AllenNLP预训练ELMo模型  
        self.elmo = elmo  
        # 隐含层  
        self.fc1 = nn.Linear(self.elmo.get_output_dim(), hidden_dim)  
        # 输出层  
        self.fc2 = nn.Linear(hidden_dim, num_class)  
        self.activate = F.relu  
  
    def forward(self, inputs, lengths):  
        elmo_output = self.elmo(inputs)  
        embeds = elmo_output['elmo_representations'][0]  
        mask = elmo_output['mask']  
  
        # 将每个序列中词的ELMo向量均值作为该序列的向量表示，作为MLP的输入  
        embeds = torch.sum(embeds * mask.unsqueeze(2), dim=1) / lengths.  
        unsqueeze(1)  
        hidden = self.activate(self.fc1(embeds))  
        output = self.fc2(hidden)  
        log_probs = F.log_softmax(output, dim=1)  
        return log_probs
```

基于语言模型的动态词向量预训练

□应用与评价

▣上下文相关的词义相似性检索

▣ELMo相比GloVe(静态词向量) 在词义消歧和近邻分析任务上都有比较好的表现

模型	词	近邻
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
ELMo	Chico Ruiz made a spectacular <u>play</u> on Alusik's grounder . . .	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent <u>play</u>
	Olivia De Havilland signed to do a Broadway <u>play</u> for Garson . . .	. . . they were actors who had been handed fat roles in a successful <u>play</u> , and had talent enough to fill the roles competently , with nice understatement

Thanks!
Q&A ?

wuxianyan@njau.edu.cn